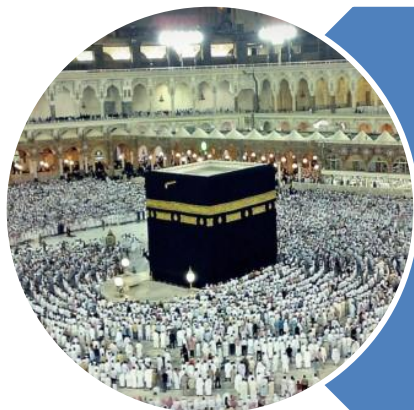


رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي أَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. **(Quran 20 : 25-28)**

Binary Search Tree (BST) Implementation

Binary Search Tree – Node Class

```
public class intBSTnode {  
    public int data;  
    public intBSTnode left;  
    public intBSTnode right;  
    public intBSTnode () {  
        left = right = null;  
    }  
    public intBSTnode (int i) {  
        this(i, null, null);  
    }  
    public intBSTnode (int i, intBSTnode n, intBSTnode p) {  
        data = i;  
        left = n;  
        right = p;  
    }  
}
```

Class – BinarySearchTree

```
public class BinarySearchTree {  
    private intBSTnode root;  
  
    // Constructor  
    public BinarySearchTree() {  
  
        root = null;  
    }  
  
    // All methods will be written here  
}
```

Binary Search Tree Operations

C

P

C

S

2

0

4

6

BST - Operations

- **Traversal**
 - Accessing/visiting all values of a tree once
- **Searching**
 - Finding a key value
- **Insertion**
 - Adding a value in a tree
- **Deletion**
 - Removing a value from tree
- **Creation**
 - Creating a tree from empty tree

BST - Traversal

- Pre Order Traversal
- In Order Traversal
- Post Order Traversal
- Depth First Search
- Breadth First Search

C

P

C

S

2

0

4

BST – Post Order

- Left - Right - **Root**
- the root is visited after both the left and right sub trees

```
public void postorder (intBSTnode p) {  
    if (p != NULL) {  
        postorder(p.left);  
        postorder(p.right);  
        System.out.print(p.data + " ");  
    }  
}
```


Binary Search Tree

Searching

(Iterative Solution)

C

P

C

S

2

0

4

BST – Searching - Iterative

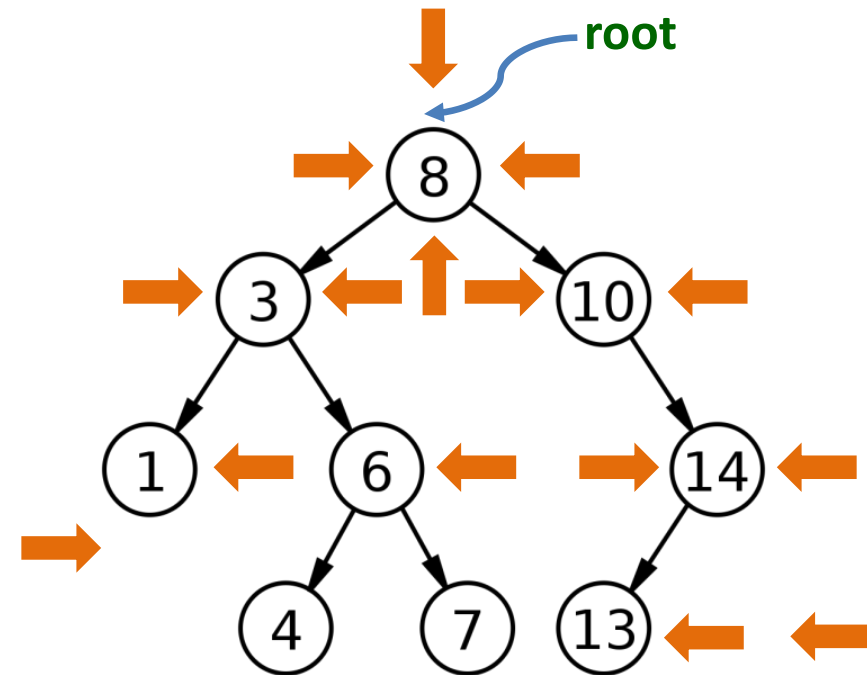
```
public intBSTnode (int key) {  
    intBSTnode current = root;  
    while (current.data != key) {  
        if (key < current.data)  
            current = current.left;  
        else  
            current = current.right;  
        if (current == null);  
        return null;  
    }  
    return current;  
}
```

"No Bonus" marks for this course anymore

C

BST – Searching - Tracing

```
public intBSTnode find(int key) {
    intBSTnode current = root;
    while (current.data != key) {
        if (key < current.data)
            current = current.left;
        else
            current = current.right;
        if (current == null);
        return null;
    }
    return current;
}
```



0

4

11

• Key = 6

Current

• Key = 13

Current

• Key = 0

Null

• Key = 20

Null

Binary Search Tree

Searching

(Recursive Solution)

C

P

C

S

2

0

4

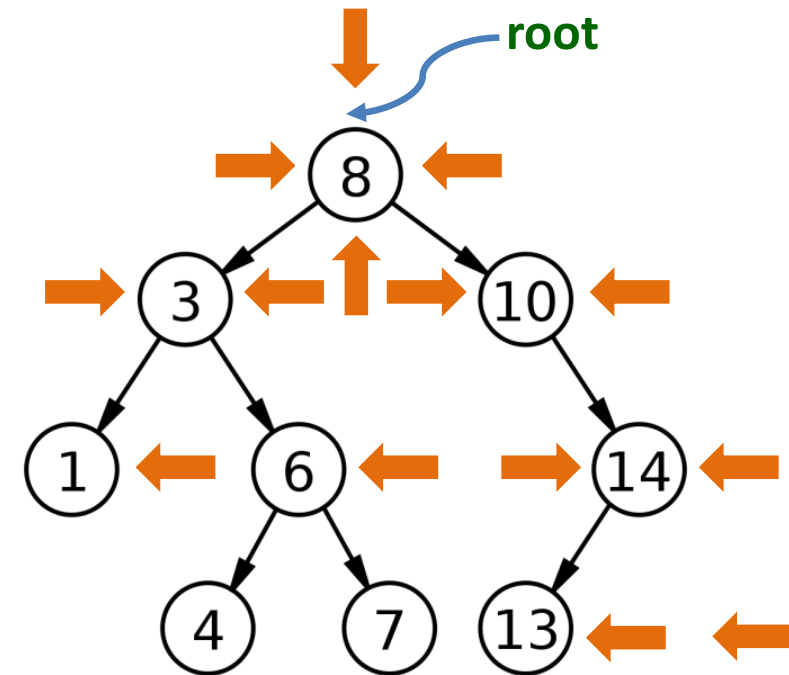
BST – Searching - Recursive

```
public boolean recurSearch(intBSTnode p, int data) {  
    if (p == null)  
        return(false);  
    if (data == p.data)  
        return(true);  
    else if (data < p.data)  
        return(recurSearch(p.left, data));  
    else  
        return(recurSearch(p.right, data));  
}
```

"No Bonus" marks for this course anymore

```
public boolean recurSearch(intBSTnode p, int data) {
    if (p == null) {
        return(false);
    }
    if (data == p.data) {
        return(true);
    }
    else if (data < p.data) {
        return(recurSearch(p.left, data));
    }
    else {
        return(recurSearch(p.right, data));
    }
}
```

Recursive



2
0
4

14

• Key = 6
True

• Key = 13
True

• Key = 0
False

• Key = 20
False

Binary Search Tree

Insertion

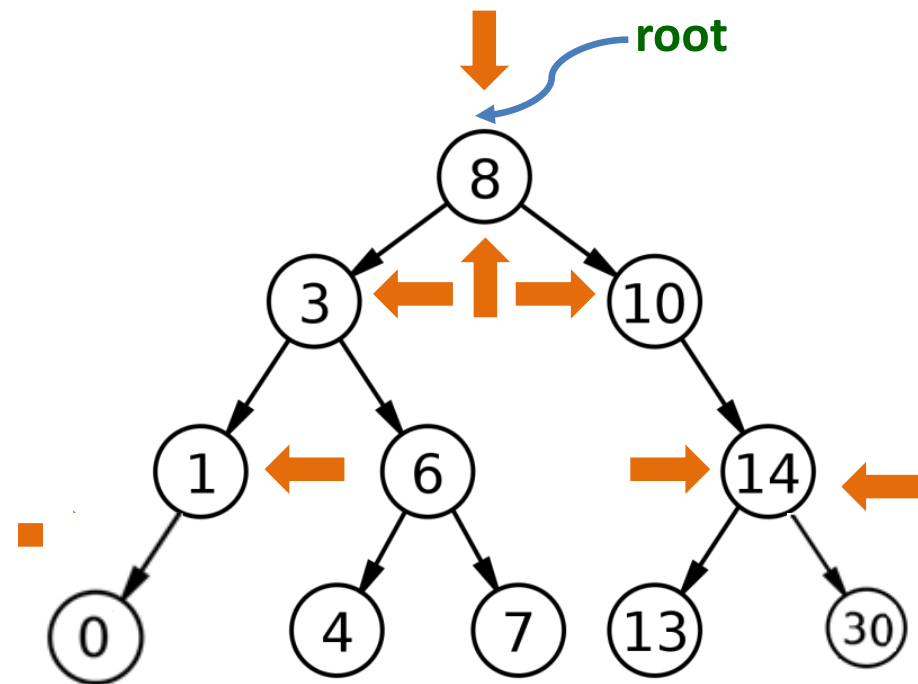
BST – Insertion

```
public intBSTnode insert(intBSTnode p, int data) {  
    if (p == null) {  
        p = new intBSTnode(data);  
    }  
    else {  
        if (data < p.data) {  
            p.left = insert(p.left, data);  
        }  
        else {  
            p.right = insert(p.right, data);  
        }  
    }  
    return p;  
}
```


C

BST – Insertion - Tracing

```
public intBSTnode insert(intBSTnode p, int data) {
    if (p == null) {
        p = new intBSTnode(data);
    }
    else {
        if (data < p.data) {
            p.left = insert(p.left, data);
        }
        else {
            p.right = insert(p.right, data);
        }
    }
    return p;
}
```



data = 0

○ Null

data = 30

○ Null

Binary Search Tree

Deletion

C

P

C

S

2

0

4

BST – Deletion

- Find node to be deleted
- Find Parent of node
- Check node is a “leaf”
- Has only Left Child
- Has only Right Child
- Find Smallest node
- Find Largest node

"No Bonus" marks for this course anymore

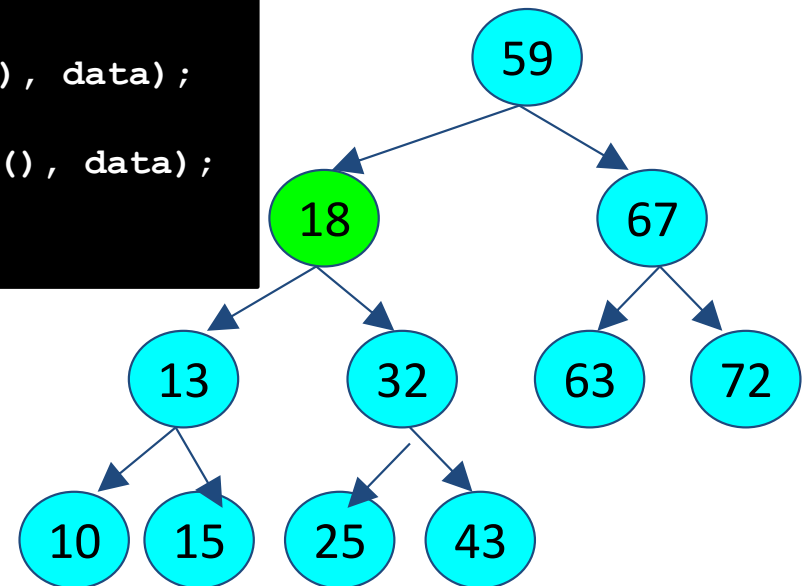
C
P
C
S
2
0
4

BST – Deletion - Find Node to Delete

```
public BSTnode findNode(int data) {  
    return findNode(root, data);  
}  
private BSTnode findNode(BSTnode p, int data) {  
    if (p == null)  
        return null;  
    else {  
        if (data == p.getData())  
            return p;  
        else if (data < p.getData())  
            return findNode(p.getLeft(), data);  
        else  
            return findNode(p.getRight(), data);  
    }  
}
```

BST – Deletion – Find Node to Delete

```
public BSTnode findNode(int data) {
    return findNode(root, data);
}
private BSTnode findNode(BSTnode p, int data) {
    if (p == null)
        return null;
    else {
        if (data == p.getData())
            return p;
        else if (data < p.getData())
            return findNode(p.getLeft(), data);
        else
            return findNode(p.getRight(), data);
    }
}
```



BST – Deletion - Find Parent Node

```
public BSTnode parent(BSTnode p) {
    return parent(root, p);
}
private BSTnode parent(BSTnode root, BSTnode p) {
    if (root == null || root == p)
        return null;
    if (root.getLeft() == p || root.getRight() == p)
        return root;
    if (p.getData() < root.getData())
        return parent(root.getLeft(), p);
    else if (p.getData() > root.getData())
        return parent(root.getRight(), p);
    else
        return null;
}
```

"No Bonus" marks for this course anymore

C

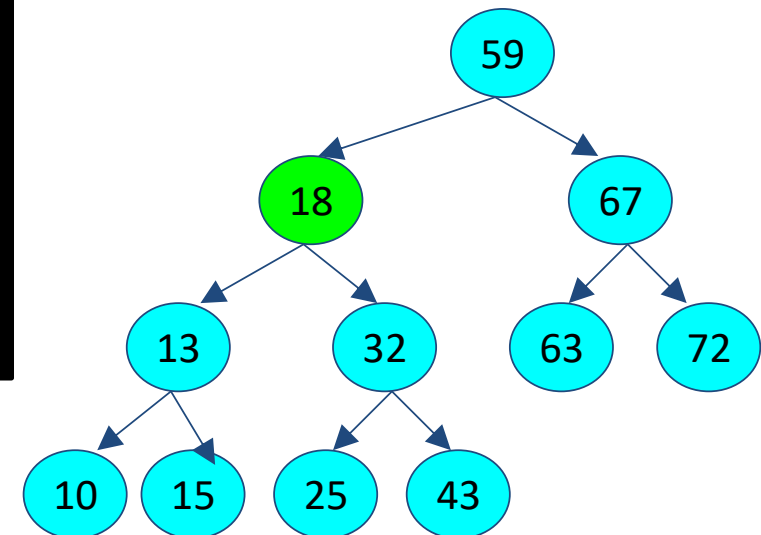
BST – Deletion - Find Parent Node

```
public BSTnode parent(BSTnode p) {
    return parent(root, p);
}
private BSTnode parent(BSTnode root, BSTnode p) {
    if (root == null || root == p)
        return null;
    if (root.getLeft() == p || root.getRight() == p)
        return root;
    if (p.getData() < root.getData())
        return parent(root.getLeft(), p);
    else if (p.getData() > root.getData())
        return parent(root.getRight(), p);
    else
        return null;
}
```

4

23

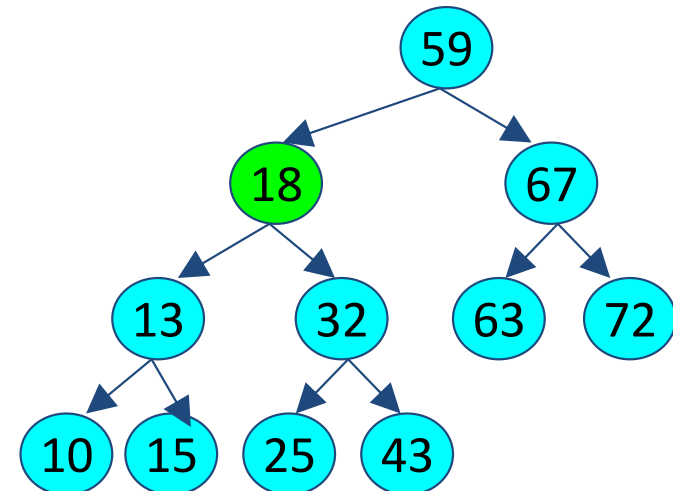
Dr. Jonathan (Yahya) Cazalas



Dr. Muhammad Umair Ramzan

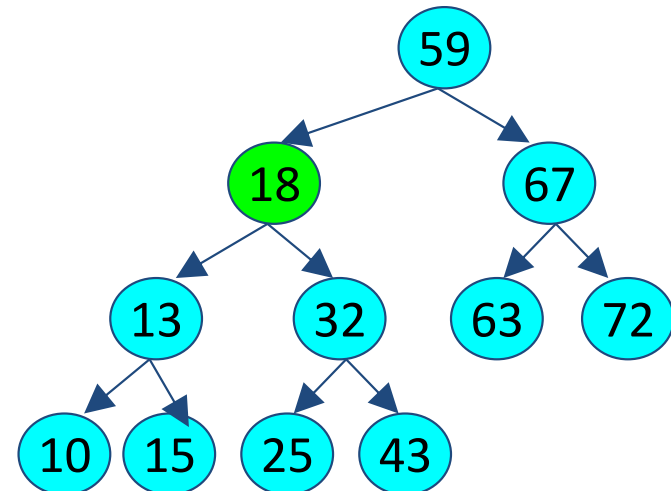
BST – Deletion – Leaf Node

```
public boolean isLeaf(BSTnode p) {
    return (p.getLeft()==null && p.getRight()==null);
}
```



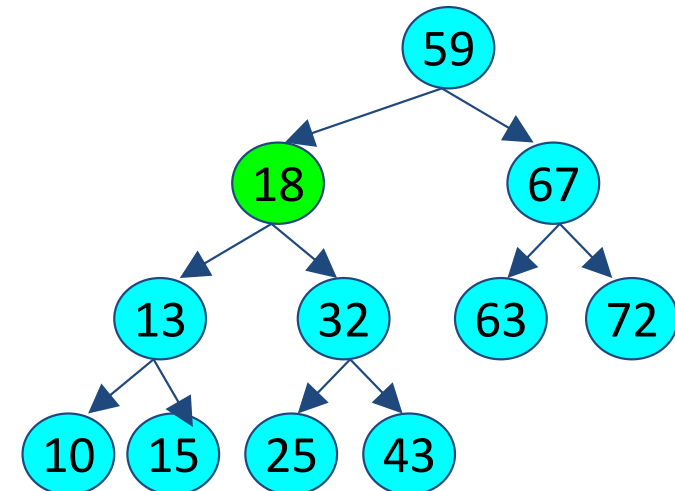
BST – Deletion – Has Left Child only

```
public Boolean hasOnlyLeftChild(BSTnode p) {
    return (p.getLeft() != null && p.getRight() == null);
}
```



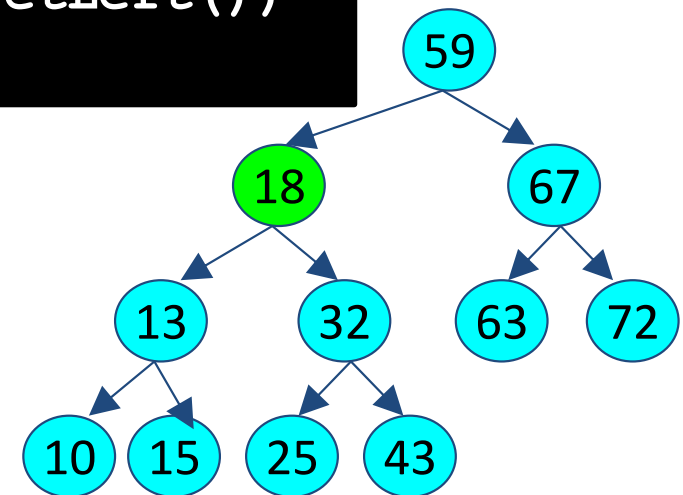
BST – Deletion – Has Right Child only

```
public Boolean hasOnlyRightChild(BSTnode p) {
    return (p.getLeft()==null && p.getRight()!=null);
}
```



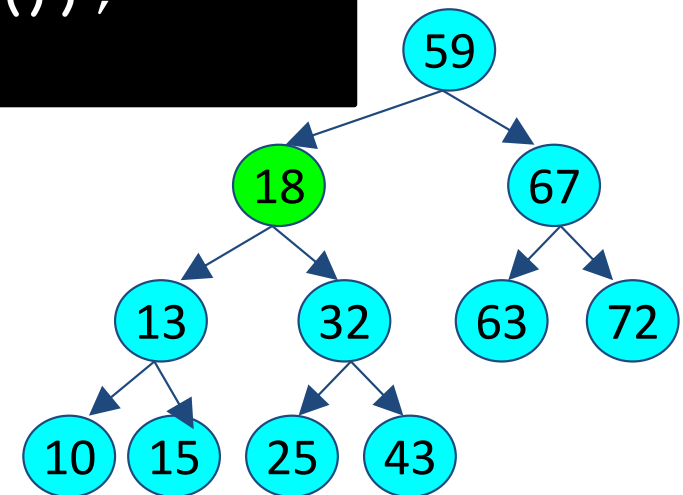
BST – Deletion – Find Smallest Node

```
public BSTnode minNode(BSTnode root) {
    if (root == null)
        return null;
    else if (root.getLeft() == null)
        return root;
    else
        return minNode(root.getLeft())
}
```



BST – Deletion – Find Largest Node

```
public BSTnode maxNode(BSTnode root) {
    if (root == null)
        return null;
    else if (root.getRight() == null)
        return root;
    else
        return maxNode(root.getRight());
}
```



C

P

C

S

2

0

4

```
public void delete(int data) {
    root = delete(root, data);
}

private BSTnode delete(BSTnode p, int data) {
    BSTnode node2delete, newnode2delete;
    BSTnode node2save, parent;
    int saveValue;
    node2delete = findNode(p, data);
    if (node2delete == null)
        return root;
    parent = parent(p, node2delete);
    if (isLeaf(node2delete)) {
        if (parent == null)
            return null;
        if (data < parent.getData())
            parent.setLeft(null);
        else
            parent.setRight(null);
        return p;
    }
}
```

```

if (hasOnlyLeftChild(node2delete)) {
    if (parent == null)
        return node2delete.getLeft();
    if (data < parent.getData())
        parent.setLeft(parent.getLeft().getLeft());
    else
        parent.setRight(parent.getRight().getLeft());
    return p;
}
if (hasOnlyRightChild(node2delete)) {
    if (parent == null)
        return node2delete.getRight();
    if (data < parent.getData())
        parent.setLeft(parent.getLeft().getRight());
    else
        parent.setRight(parent.getRight().getRight());
    return p;
}
    
```

C

P

C

S

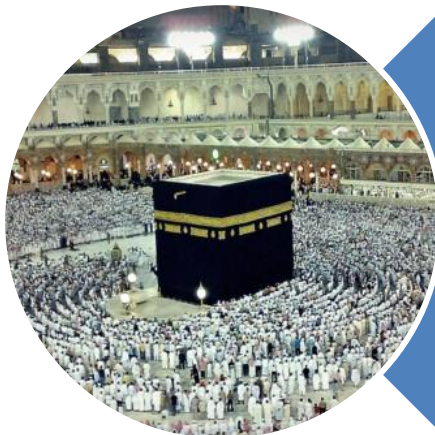
2

0

4

BST – Deletion

```
newnode2delete = minNode(node2delete.getRight());  
saveValue = newnode2delete.getData();  
p = delete(p, saveValue);  
node2delete.setData(saveValue);  
return p;  
}
```

Allah (Alone) is Sufficient for us,
and He is the Best Disposer of
affairs (for us). **(Quran 3 : 173)**